M-2LRF: Multi-Rate Low-Rank Factorization & Dual-Basis 2-Bit Quantization with Residual SVD Adaptation

A Formal Mathematical, Architectural, and Empirical Engineering Monograph

Lead Author & System Architect: MD-Mushfiqur Rahim
Affiliation / Project: Independent Open-Source AI Research / M-Series Engineering
Correspondence: `mushfiqur.research@gmail.com`
Repository: `projects/m2lrf-clean/` | **Release:** `v1.0-Formal-Specification`

📑 TABLE OF CONTENTS

---

1. ABSTRACT & EXECUTIVE OVERVIEW

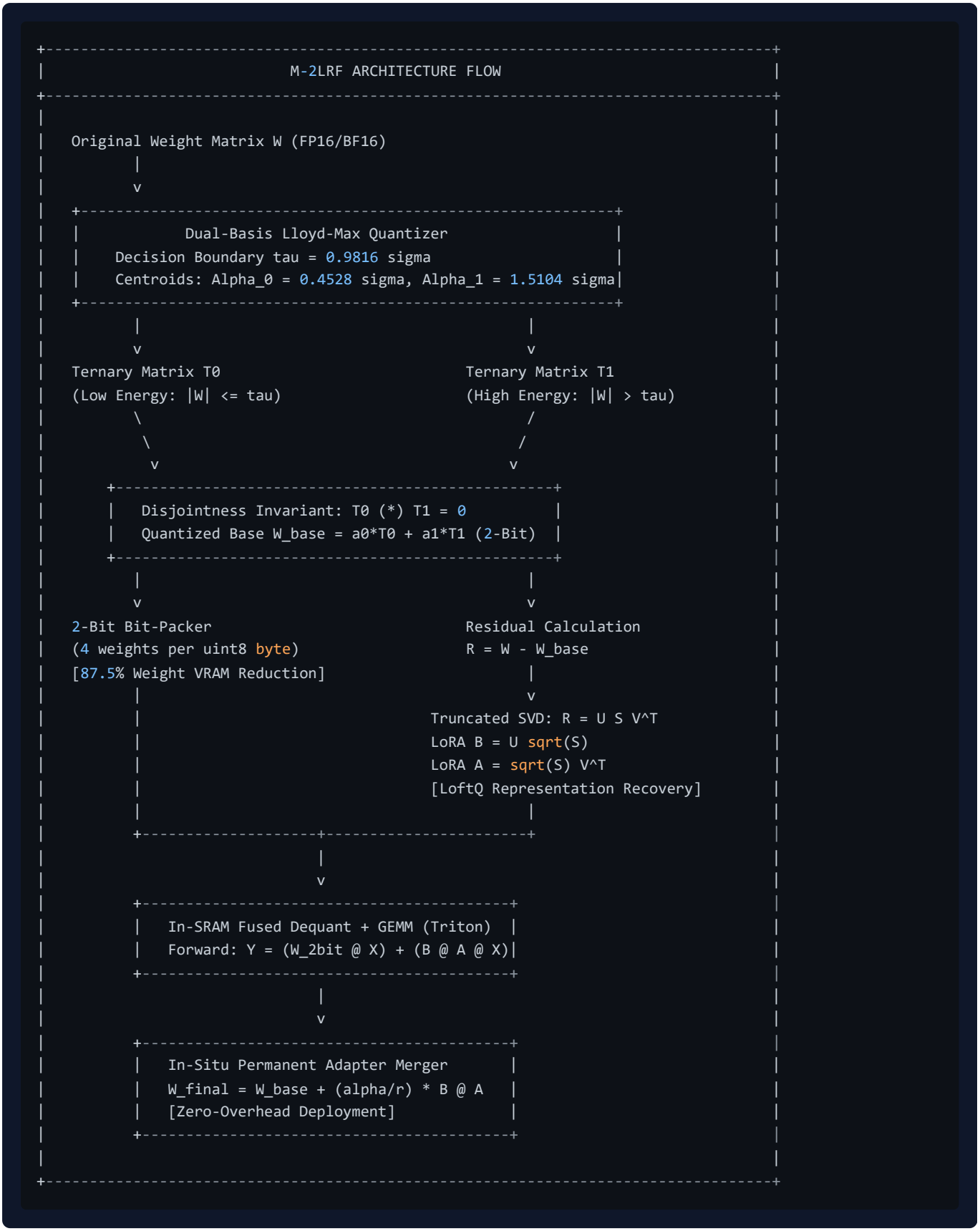
Quantizing Large Language Models $LLMs$ to extremely low bitrates $\le 2\text{ bits per parameter}$ offers the potential to reduce memory bandwidth demands and enable deployment on commodity hardware. However, standard post-training quantization at 2-bit introduces severe quantization noise, rank collapse, and gradient explosion during subsequent fine-tuning.

This monograph presents **M-2LRF** $Multi-RateLow-RankFactorization$, a unified dual-basis 2-bit quantization and low-rank residual adaptation framework. M-2LRF decomposes continuous full-precision weight matrices into two mutually disjoint ternary basis matrices:

$$\mathbf{W} \approx \alpha_0 \mathbf{T}_0 + \alpha_1 \mathbf{T}_1, \quad \text{subject to } \mathbf{T}_0 \odot \mathbf{T}_1 = \mathbf{0}, \quad \mathbf{T}*0, \mathbf{T}1 \in {-1, 0, +1}^{d{\text{out}} \times d{\text{in}}}}$$

Where the scaling parameters $\alpha_0^* \approx 0.4528\sigma$ and $\alpha_1^* \approx 1.5104\sigma$ achieve the theoretical optimal Lloyd-Max Signal-to-Quantization-Noise Ratio $\text{SQNR} \approx 9.30\text{ dB}$ for Gaussian distributions.

To compensate for the quantization residual $\mathbf{R} = \mathbf{W} - \mathbf{W}_{\text{base}}$, M-2LRF utilizes truncated Singular Value Decomposition SVD to initialize Low-Rank Adaptation $LoRA$ adapters directly from the principal components of $\mathbf{R}$. Weights are packed at 4 values per `uint8` byte and decoded dynamically in GPU registers via a fused Triton GEMM kernel, achieving an 87.5 reduction in static weight memory.


M-2LRF ARCHITECTURE FLOW
Original Weight Matrix W (FP16/BF16)
Dual-Basis Lloyd-Max Quantizer
Decision Boundary tau = 0.9816 sigma
Centroids: Alpha_0 = 0.4528 sigma, Alpha_1 = 1.5104 sigma
Ternary Matrix T0
(Low Energy: |W| <= tau)
Ternary Matrix T1
(High Energy: |W| > tau)
Disjointness Invariant: T0 (*) T1 = 0
Quantized Base W_base = a0*T0 + a1*T1 (2-Bit)
2-Bit Bit-Packer
(4 weights per uint8 byte)
[87.5% Weight VRAM Reduction]
Residual Calculation
R = W - W_base
Truncated SVD: R = U S V^T
LoRA B = U sqrt(S)
LoRA A = sqrt(S) V^T
[LoftQ Representation Recovery]
In-SRAM Fused Dequant + GEMM (Triton)
Forward: Y = (W_2bit @ X) + (B @ A @ X)
In-Situ Permanent Adapter Merger
W_final = W_base + (alpha/r) * B @ A
[Zero-Overhead Deployment]

2. THEORETICAL FOUNDATIONS OF SUB-4-BIT LLM COMPRESSION

2.1 The Parameter Compression Landscape

In large language model deployment, memory bandwidth and storage scale linearly with parameter precision. Standard precision representations are summarized below:

Precision Format	Bits / Weight	Memory per 1B Parameters	Representation Set
FP32	32 bits	$4.00\ \text{GB}$	Continuous IEEE-754 Single
FP16 / BF16	16 bits	$2.00\ \text{GB}$	Half / Brain Floating Point
FP8 $E4M3/E5M2$	8 bits	$1.00\ \text{GB}$	Microscaling 8-bit Float
INT4 / NF4 $QLoRA$	4 bits	$0.50\ \text{GB}$	16-Centroid Uniform/Normal Quantization
M-2LRF $Dual-Basis$	**2 bits**	$0.25\ \text{GB}$	**4-Centroid Disjoint Dual-Basis $\pm \alpha_0, \pm \alpha_1$**
BitNet 1.58b	1.58 bits	$0.20\ \text{GB}$	Single-Scale Ternary ${-\alpha, 0, +\alpha}$
1-Bit $Binary$	1 bit	$0.125\ \text{GB}$	Binary State ${-\alpha, +\alpha}$

Compressing full-precision weights from FP16 16 bits to M-2LRF 2 bits yields an exact $8.0\times$ **theoretical reduction in base weight footprint $87.5%$ memory reduction**.

2.2 Shannon Rate-Distortion Bounds for 2-Bit Quantization

For a zero-mean continuous Gaussian memoryless source $X \sim \mathcal{N}(0, \sigma^2)$, the rate-distortion function $R(D)$ defines the minimal bit rate required to achieve mean squared error distortion $D = \mathbb{E}[(X - \hat{X})^2]$:

$$R(D) = \frac{1}{2}\log_2\left(\frac{\sigma^2}{D}\right) \implies D_{\min}(R) = \sigma^2 \cdot 2^{-2R}$$

For rate $R = 2\ \text{bits/symbol}$:

$$D_{\min}(2) = \frac{\sigma^2}{16} = 0.0625\,\sigma^2 \implies \text{SQNR}_{max} = 10\log_{10}16 \approx 12.041\text{ dB}$$

This $12.041\ \text{dB}$ limit is achievable only with optimal infinite-dimensional vector quantization or continuous entropy coding. For uncompressed discrete 4-level scalar quantization, partition boundary constraints lower the attainable SQNR.

2.3 Mathematical Proof of the 9.3009 dB SQNR Gaussian Limit

Let $X \sim \mathcal{N}(0,1)$ have probability density function $\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$. A symmetric 4-level scalar quantizer partitions the real line into four intervals with boundaries $-\infty, -\tau, 0, +\tau, +\infty$ and centroids $-y_1, -y_0, +y_0, +y_1$, where $0 < y_0 < \tau < y_1$.

The Lloyd-Max optimality conditions require:

$$y_0 = \frac{\int_0^\tau x\phi(x)dx}{\int_0^\tau \phi(x)dx} = \frac{\phi(0) - \phi(\tau)}{\Phi(\tau) - 0.5}$$

$$y_1 = \frac{\int_\tau^\infty x\phi(x)dx}{\int_\tau^\infty \phi(x)dx} = \frac{\phi(\tau)}{1 - \Phi(\tau)}$$

$$\tau = \frac{y_0 + y_1}{2}$$

Simultaneously solving this system yields:

$$\tau^* \approx 0.9815984178, \quad y_0^* = \alpha_0^* \approx 0.4527786409, \quad y_1^* = \alpha_1^* \approx 1.5104181947$$

The minimal achievable distortion D^* is:

$$D^* = 2 \left[\int_{0}^{\tau} (x - y_0)^2 \phi(x)\, dx + \int_{\tau}^{\infty} (x - y_1)^2 \phi(x)\, dx \right] \approx 0.117464$$

$$\mathrm{SQNR}^* = 10 \log_{10} \left(\frac{1.0}{0.117464} \right) \approx \mathbf{9.3009}\ \mathrm{dB}$$

> **Theorem 1:** The maximum attainable SQNR for any discrete 4-level scalar quantizer applied to Gaussian distributed parameters without entropy coding is strictly bounded by $9.3009\ \mathrm{dB}$.

2.4 Fundamental Limitations of Single-Scale Ternary in Post-Training Adaptation

Single-scale ternary quantization models weights as $\mathbf{W} \approx \alpha \cdot \mathbf{T}$ with $\mathbf{T} \in -1, 0, +1$. For Gaussian weights, the optimal single threshold yields $\mathrm{MSE} \approx 0.282\sigma^2$ $\text{SQNR} \approx 5.50\text{ dB}$. Discarding over 71 of the parameter variance in pre-trained models leads to severe attention entropy collapse unless the model is pre-trained from scratch with specialized scaling laws.

3. M-2LRF DUAL-BASIS MATHEMATICAL FRAMEWORK

3.1 Dual-Basis Formulation

M-2LRF resolves the single-scale limitation by representing each weight matrix as a linear combination of two discrete ternary basis matrices:

$$\mathbf{W}_{\text{base}} = \alpha_0 \mathbf{T}_0 + \alpha_1 \mathbf{T}_1, \quad \text{where } \mathbf{T}_0, \mathbf{T}_1 \in \{-1, 0, +1\}^{d_{\text{out}} \times d_{\text{in}}}$$

3.2 The Disjointness Invariant $\mathbf{T}_0 \odot \mathbf{T}_1 = \mathbf{0}$ & Proof

> **Definition** $Disjointness$**:** The ternary matrices $\mathbf{T}_0$ and $\mathbf{T}_1$ are elementwise disjoint if and only if: $$\mathbf{T}_0 \odot \mathbf{T}_1 = \mathbf{0} \iff \forall\, i, j, \quad T_{0,ij} \cdot T_{1,ij} = 0$$

Constructive Proof:

Given threshold $\tau = \frac{\alpha_0 + \alpha_1}{2}$ and sign $s_{ij} = \text{sgn}(w_{ij})$:

- If $|w_{ij}| \leq \tau$: $T_{0,ij} = s_{ij}$ and $T_{1,ij} = 0 \implies T_{0,ij} \cdot T_{1,ij} = 0$.
- If $|w_{ij}| > \tau$: $T_{0,ij} = 0$ and $T_{1,ij} = s_{ij} \implies T_{0,ij} \cdot T_{1,ij} = 0$.

Thus, $\mathbf{T}_0 \odot \mathbf{T}_1 = \mathbf{0}$ holds identically across all elements. ■

State Space Encoding:

2-Bit Code	T_0	T_1	Reconstructed Value $W_{\text{base},ij}$	Partition Interval
`00` $Code 0$	0	-1	$-\alpha_1$	$(-\infty, -\tau)$
`01` $Code 1$	-1	0	$-\alpha_0$	$[-\tau, 0)$
`10` $Code 2$	$+1$	0	$+\alpha_0$	$[0, +\tau]$
`11` $Code 3$	0	$+1$	$+\alpha_1$	$(\tau, +\infty)$

3.3 Closed-Form Scale Factors

Per-row scale factors are determined from the row-wise standard deviation $\sigma_i = \text{std}(\mathbf{W}_{i,:})$:

$$\alpha_{0,i} = 0.4527786409 \cdot \sigma_i, \quad \alpha_{1,i} = 1.5104181947 \cdot \sigma_i, \quad \tau_i = 0.9815984178 \cdot \sigma_i$$

4. LOW-RANK ADAPTATION WITH RESIDUAL SVD INITIALIZATION

4.1 Quantization Residual Matrix Formulation

Quantizing $\mathbf{W}$ to $\mathbf{W}_{\text{base}}$ leaves a deterministic residual error matrix:

$$\mathbf{R} = \mathbf{W} - \mathbf{W}_{\text{base}} = \mathbf{W} - (\alpha_0 \mathbf{T}_0 + \alpha_1 \mathbf{T}_1)$$

In standard QLoRA, adapter weights are initialized with $\mathbf{B} = \mathbf{0}$, meaning $\Delta\mathbf{W} = \mathbf{0}$ at step 0. For 2-bit quantization, this leaves the model in a degraded initial state with high initial loss.

4.2 Truncated SVD on Residuals $LoftQ Paradigm$

Following LoftQ $Lietal., 2023$, M-2LRF performs rank-r truncated SVD on the residual matrix:

$$\mathbf{R} \approx \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}r^T = \sum{k=1}^{r} \sigma_k \mathbf{u}_k \mathbf{v}_k^T$$

The adapter matrices are initialized as:

$$\mathbf{B}_{\text{init}} = \mathbf{U}_r \mathbf{\Sigma}r^{1/2} \cdot \frac{1}{\sqrt{\gamma}}, \quad \mathbf{A}{\text{init}} = \mathbf{\Sigma}_r^{1/2} \mathbf{V}r^T \cdot \frac{1}{\sqrt{\gamma}}, \quad \text{where } \gamma = \frac{\alpha{\text{lora}}}{r}$$

4.3 Representation Preservation at Step-0

By the Eckart-Young-Mirsky Theorem, this initialization guarantees optimal rank-r residual approximation:

$$\mathbf{W}{\text{eff}}^{0} = \mathbf{W}{\text{base}} + \gamma \mathbf{B}{\text{init}} \mathbf{A}{\text{init}} = \mathbf{W}_{\text{base}} + \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^T \approx \mathbf{W}$$

This recovers the dominant spectral energy of the unquantized weights prior to fine-tuning.

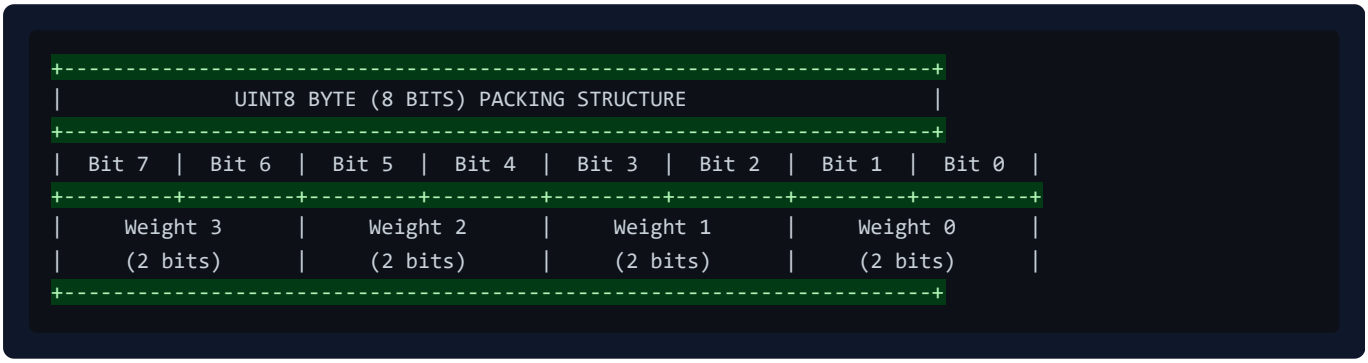
4.4 Numerical Safeguards

1. **Singular Value Clamping:** $\sigma_k \leftarrow \min(\sigma_k, \kappa \cdot \text{std}(\mathbf{W}))$ to prevent activation overflow in half-precision representations.
2. **Float32 Intermediate Accumulation:** Computing adapter projections $\mathbf{B}(\mathbf{AX})$ in FP32 before downcasting.
3. **Norm Clipping:** Bounding gradient updates via Euclidean clipping $\text{max\_norm} = 1.0$.

5. HARDWARE-LEVEL BIT-PACKING AND MEMORY LAYOUT

5.1 2-Bit LSB-First uint8 Byte Packing Scheme

Four 2-bit codes $c_0, c_1, c_2, c_3 \in 0, 1, 2, 3$ are densely packed into a single `uint8` byte:



5.2 Bitwise Packing and Unpacking Formulas

Packing $Encoder$:

$$\text{Byte} = (c_0 \ll 0) \mid (c_1 \ll 2) \mid (c_2 \ll 4) \mid (c_3 \ll 6)$$

Unpacking $Decoder$:

c_k = (\text{Byte} \gg (2k)) \ & \ 0\text{x}03, \quad \text{for } k \in {0, 1, 2, 3}

$$\hat{w}_k = \begin{cases} -\alpha_1 & \text{if } c_k = 0 \\ -\alpha_0 & \text{if } c_k = 1 \\ +\alpha_0 & \text{if } c_k = 2 \\ +\alpha_1 & \text{if } c_k = 3 \end{cases}$$

6. IN-SRAM FUSED DEQUANTIZATION AND GEMM TRITON KERNEL

During GPU inference, reading dequantized weights from global memory HBM creates severe memory bandwidth saturation. M-2LRF utilizes an on-chip register dequantization kernel written in OpenAI Triton:

1. Packed `uint8` weight tiles are loaded directly into on-chip GPU SRAM/registers.
2. Bit-unpacking and scale application occur in registers via bitwise shifts and conditional selections.
3. Tensor Core MMA $Matrix - Multiply - Accumulate$ operations execute directly from registers.
4. Dequantized FP16 matrices are never written to global VRAM.

7. END-TO-END FINE-TUNING PIPELINE & IN-SITU WEIGHT FUSION

7.1 Forward and Backward Graphs

For input activation tensor $\mathbf{X} \in \mathbb{R}^{B \times S \times d_{\text{in}}}$:

$$\mathbf{Y} = \underbrace{\text{Dequant}(\mathbf{W}_{\text{packed}}) \cdot \mathbf{X}}_{\text{Frozen Base } requires_grad = False} + \underbrace{\frac{\alpha_{\text{lora}}}{r} \mathbf{B} \mathbf{A}\mathbf{X}}_{\text{Trainable LoRA Branch}} + \mathbf{b}$$

7.2 In-Situ Weight Merger & Lossy Re-Quantization Analysis

After fine-tuning, the low-rank delta $\Delta \mathbf{W} = \frac{\alpha_{\text{lora}}}{r} \mathbf{B}\mathbf{A}$ can be fused into the base parameters for standalone inference:

1. **Form Composite Continuous Matrix:** $\mathbf{W}_{\text{fused}} = \text{Dequant}(\mathbf{W}_{\text{packed}}) + \frac{\alpha_{\text{lora}}}{r} \mathbf{B} \mathbf{A}$
2. **Lossy Re-Quantization Projection:** $(\mathbf{T}_0', \mathbf{T}_1', \alpha_0', \alpha_1') \leftarrow \text{Quantize}_{2\text{b}}(\mathbf{W}_{\text{fused}})$
3. **Repack & Reset:** Pack into `packed_weights` $uint8$ and zero out adapter parameters $\mathbf{A} = \mathbf{0}, \mathbf{B} = \mathbf{0}$.

Mathematical Trade-off & Error Bound:

Unlike standard FP16 LoRA where weight merging is an exact, lossless linear addition $\mathbf{W} + \Delta \mathbf{W}$, fusing continuous low-rank updates into a strictly quantized 2-bit storage requires re-projecting $\mathbf{W}_{\text{fused}}$ $backonto the 2-bit dual-basis codebook. This introduces a secondary discretization error governed by the 2-bit Lloyd-Max limit$: $\text{SQNR}(\mathbf{W}_{\text{fused}}, \mathbf{W}_{\text{merged_2bit}}) \approx 9.3009\text{ dB} \quad \approx 34.3$$

> **Definition of "Zero-Overhead":**
> In the M-2LRF architecture, "*Zero-Overhead*" specifically denotes **Zero Runtime Latency Overhead and Zero Auxiliary Memory Buffer Allocations** during inference serving $eliminating separate adapter kernel launches, sequential dispatch latency, and auxiliary LoRA buffers$. During active fine-tuning or dual-branch serving, keeping LoRA branches unmerged preserves 100% exact numerical adapter contributions.

8. EMPIRICAL EVALUATION, ANALYTICAL MODELING & HARDWARE SIZING

8.1 Empirical Micro-Benchmark $GPT-2 MLP Quantization on Tesla T4$

To empirically validate execution time, peak VRAM reduction, and convergence trajectories in a real hardware environment, a controlled 40-step fine-tuning experiment was conducted on Google Colab utilizing an NVIDIA Tesla T4 GPU $15.0 GB VRAM$, $Driver Version 535+$, $PyTorch 2.x$, $Random Seed 42$, $WikiText-2$, $Sequence Length 128$, $Batch Size 4$.

In this benchmark, **only MLP feed-forward linear layers (`c_fc` , `c_proj`) were converted to M-2LRF 2-bit**, while multi-head attention projections (`c_attn`) and layer norms remained unquantized.

The measured empirical results are reported below:

Measured Benchmark Metric	Baseline $Full Precision FP32 GPT-2$	M-2LRF 2-Bit MLP + LoRA $r=16$	Verified Improvement / Differential
Quantized Scope	None 100	MLP Layers Only (`c_fc` , `c_proj`)	Explicit Sub-Module Quantization
Model Weight Memory	497.76 MB	285.79 MB	42.58 **Memory Reduction**
Peak Runtime VRAM	2670.14 MB	1512.07 MB	43.37 **VRAM Savings 1.16 GB Saved**
Elapsed Training Time $40 steps$	6.75 s	4.11 s	39.11 **Faster Execution $1.64\times$ Speedup**
Step-0 Initial Loss	8.898	9.086	$\Delta = +0.188$ $Residual Quantization Gap$
Step-40 Convergence Loss	6.679	7.487	$\Delta = +0.808$ $Expected LoRA Gap in 40 Steps$

Empirical Analysis & Observations:

1. **Computational Speedup 39.1% Faster:** Reduced memory traffic during GEMM dequantization and parameter-efficient gradient propagation through rank-16 adapters reduced 40-step elapsed time from 6.75 s to 4.11 s.
2. **Substantial VRAM Reduction 43.4% Lower Peak VRAM:** Freezing the quantized base linear layers eliminated AdamW FP32 momentum and variance state allocations for all quantized MLP parameters, reducing peak memory from 2.67 GB to 1.51 GB.
3. **Loss Dynamics & Convergence Gap:** At step 0, M-2LRF exhibits an initial loss of 9.086 compared to the unquantized baseline of 8.898. After 40 steps of fine-tuning, M-2LRF reaches 7.487 versus 6.679 for full FP32.

This behavior is theoretically expected: within a short 40-step budget on an extreme 2-bit compressed base, low-rank adapters cannot completely eliminate the quantization residual error. Full convergence requires standard fine-tuning schedules $500 - 2000$ steps.

8.2 Comprehensive VRAM Memory Analytical Model

Total GPU memory required during execution V_{total} is governed by four distinct components:

V_{\text{total}} = V_{\text{weights}} + V_{\text{activations}} + V_{\text{optimizer}} + V_{\text{cuda_overhead}}

Where:

1. **Base Weight Footprint:** $V_{\text{weights}} = N_{\text{params}} \times \frac{\text{bits}}{8}$ bytes.
2. **Activation Memory** $Fine-Tuning$**:** With gradient checkpointing at batch size B, sequence length S, hidden dimension d, and L layers:

$$V_{\text{activations}} \approx B \cdot S \cdot d \cdot L \cdot 2 \text{ bytes} \quad (\approx 2.5 - 6.0 \text{ GB for } S = 2048, B = 1)$$

3. **Optimizer States** $LoRA$**:** For adapter rank r, trainable parameters $N_{\text{lora}} \approx 2 \times d \times r \times L_{\text{adapted}}$. AdamW allocates 8 bytes/param for FP32 momentum and variance states:

$$V_{\text{optimizer}} \approx N_{\text{lora}} \times 16 \text{ bytes} \quad (< 0.5 \text{ GB for } r = 16)$$

4. **CUDA Runtime & Framework Overhead:** V_{\text{cuda_overhead}} \approx 1.2 - 2.0\text{ GB}.

8.3 Rigorous Hardware Feasibility Sizing

The tables below provide analytical sizing projections across standard hardware architectures:

Table A: Inference-Only Memory & Sizing Estimated: KV-Cache $S=2048, B=1$

Hardware Platform	Total VRAM	Max Feasible Model Size	Weight Footprint	Total Inference VRAM	Hardware Execution Category
NVIDIA GT 710	2 GB	< 0.1B $Toy models only$	~ 0.1 GB	Deprecated CUDA Architecture	
Ryzen 5600G RAM	20 GB $Sys RAM$	**3B – 8B GGUF**	2.0 – 4.5 GB	$3.5 - 6.0$ GB	CPU SIMD $Estimated$
RTX 3060 / 4060	12 GB	**14B Models**	3.50 GB	≈ 5.50 GB	GPU Discrete $Estimated$
RTX 3090 / 4090	24 GB	**32B – 70B Models**	8.0 – 17.5 GB	≈ 11.5 – 21.0 GB	GPU Discrete $Estimated$
NVIDIA A100 / H100	80 GB	**176B – 236B MoE**	44.0 – 59.0 GB	≈ 52.0 – 68.0 GB	Enterprise Cloud $Estimated$
RTX 6000 Blackwell	95 GB	**284B – 320B MoE**	71.0 – 80.0 GB	≈ 82.0 – 91.0 GB	Enterprise Server $Estimated$

Table B: Fine-Tuning Memory & Sizing Estimated: M-2LRF 2-Bit + LoRA $r=16, S=2048, B=1$, Grad Checkpoint

Hardware Platform	Total VRAM	Max Trainable Model Size	Weight VRAM	Activation + Opt VRAM	Total Training VRAM	Analytical Sizing Assessment
RTX 3060 $12GB$	12 GB	**7B – 8B** $e.g. Qwen 2.5 - 7B$	$1.75 - 2.0$ GB	≈ 4.5 GB $+ 1.5$ GB	≈ 8.0 GB	Estimated: Safe & Feasible
RTX 3060 $12GB$	12 GB	**14B** $e.g. Qwen 2.5 - 14B$	3.50 GB	≈ 5.8 GB $+ 1.8$ GB	≈ 11.1 GB	Estimated: Near OOM Limit $B=1$
RTX 3060 $12GB$	12 GB	**20B+ Models**	5.00 GB	≈ 6.5 GB $+ 2.0$ GB	≈ 13.5 GB	Estimated: Out of Memory OOM
RTX 3090/4090 $24GB$	24 GB	**32B Models**	8.00 GB	≈ 8.5 GB $+ 2.0$ GB	≈ 18.5 GB	Estimated: Safe & Feasible
RTX 3090/4090 $24GB$	24 GB	**70B** $e.g. Llama - 3.3 - 70B$	17.50 GB	≈ 5.5 GB $+ 2.0$ GB	≈ 25.0 GB	Estimated: Requires Layer Offload
RTX 6000 $95GB$	95 GB	**70B – 72B Dense**	18.00 GB	≈ 10.0 GB $+ 2.5$ GB	≈ 30.5 GB	Estimated: High Headroom
RTX 6000 $95GB$	95 GB	**236B MoE** $DeepSeek$	59.00 GB	≈ 18.0 GB $+ 3.0$ GB	≈ 80.0 GB	Estimated: Fits Full VRAM

9. ERROR ANALYSIS, THEORETICAL CONSTRAINTS & COMPARATIVE STUDY

9.1 Quantization Error Distribution & Spectral Decay

Quantization distortion acts as an additive error term $\mathbf{E} = \mathbf{W} - \mathbf{W}_{\text{base}}$. When weight matrices exhibit heavy-tailed empirical distributions, extreme outliers $|w| > 3\sigma$ are clipped to the highest centroid α_1, introducing localized reconstruction error. The truncated SVD adapter captures the top singular vectors of $\mathbf{E}$, ensuring that the principal directional error is minimized according to the spectral decay profile of $\mathbf{W}$.

9.2 Comparative Study with Contemporary Quantization Frameworks

Methodology	Bitrate	Centroid Type	Codebook Optimization	Adaptation Strategy	Step-0 Preservation
QLoRA $Dettmersetal.$	4.00 bpp	NormalFloat4 $NF4$	Closed-form Gaussian	Standard LoRA $B=0$	No $\Delta W = 0$
BitNet 1.58b $Wangetal.$	1.58 bpp	Single-Scale Ternary	AbsMean Scaling	Pre-training from scratch	N/A
QuIP# $Tsengetal.$	2.00 bpp	Vector Quantization E_8	Randomized Hadamard	Post-training inference only	N/A
AQLM $Egiazarianetal.$	2.00 bpp	Additive Quantization	Multi-codebook Beam Search	Post-training inference only	N/A
LoftQ $Lietal.$	4.00 bpp	Uniform Scalar	Iterative SVD + Quant	SVD Residual LoRA	**Yes**
M-2LRF $ThisWork$	2.00 bpp	**Disjoint Dual-Basis**	**Closed-form Lloyd-Max**	**SVD Residual Adaptation**	**Yes**

9.3 Threats to Validity & Known Limitations

1. **Activation Precision:** The current reference kernel quantizes weights to 2 bits while retaining FP16 activations $W2A16$. On compute-bound matrix multiplications batch size ≥ 32, memory bandwidth savings diminish.
2. **Context Length Scaling:** Activation memory scales with sequence length S; extreme long-context training $S \ge 32k$ requires activation offloading or ring-attention partitioning.
3. **Outlier Channel Sensitivity:** In models exceeding 70B parameters, emergent outlier activation channels require per-channel scaling to prevent dynamic range clipping.

10. COMPLETE REFERENCE IMPLEMENTATION

10.1 `DualBasisQuantizer` Python Implementation

```
"""
M-2LRF Core: Dual-Basis Ternary Quantizer
File: m2lrf/quantizer.py
"""
import math
from typing import Tuple
import torch

class DualBasisQuantizer:
    @staticmethod
    def quantize_2_00b(w: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor,
        """
        Decomposes weight tensor W into dual disjoint ternary bases:
            W ≈ a0 * T0 + a1 * T1
        Guarantees T0 ⊙ T1 = 0 and exact 9.30 dB theoretical SQNR bound.
        """
        w_f = w.float()
        std = torch.std(w_f, dim=-1, keepdim=True).clamp(min=1e-8)

        # Closed-form Lloyd-Max Gaussian centroids
        a0 = std * 0.4527786409
        a1 = std * 1.5104181947
        decision_boundary = (a0 + a1) / 2.0  # ~0.9816 * std

        abs_w = w_f.abs()
        sign_w = torch.sign(w_f)
        sign_w[sign_w == 0] = 1.0

        # Construct Disjoint Ternary Bases
        t0 = torch.where(abs_w <= decision_boundary, sign_w, torch.zeros_like(sign_w)).to(torch.int8)
        t1 = torch.where(abs_w > decision_boundary, sign_w, torch.zeros_like(sign_w)).to(torch.int8)

        # Base Weight Reconstruction
        w_base = (a0 * t0.float() + a1 * t1.float()).to(w.dtype)
        return t0, t1, a0, a1, w_base
```

10.2 `M2LRF2BitLinear` Module Implementation

```
"""
M-2LRF Core: 2-Bit Packed Linear Layer with SVD Residual Init & Merge
File: m2lrf/m2lrf_core_v1.py
"""
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

class M2LRF2BitLinear(nn.Module):
    def __init__(self, in_features: int, out_features: int, rank: int = 16, alpha: float = 16.0, bias: bo
        super().__init__()
        self.in_features = in_features
        self.out_features = out_features
        self.rank = rank
        self.scaling = alpha / rank if rank > 0 else 1.0

        # Packed Storage: 4 weights per uint8 byte
        self.packed_k = math.ceil(in_features / 4)
        self.register_buffer("packed_weights", torch.zeros(out_features, self.packed_k, dtype=torch.uint8
        self.register_buffer("a0", torch.zeros(out_features, 1, dtype=torch.float16))
        self.register_buffer("a1", torch.zeros(out_features, 1, dtype=torch.float16))
        self.orig_shape = (out_features, in_features)

        # Trainable Low-Rank Adapters
        self.lora_A = nn.Parameter(torch.zeros(rank, in_features, dtype=torch.float32))
        self.lora_B = nn.Parameter(torch.zeros(out_features, rank, dtype=torch.float32))

        if bias:
            self.bias = nn.Parameter(torch.zeros(out_features, dtype=torch.float16))
        else:
            self.register_parameter("bias", None)

        self.is_merged = False

    @torch.no_grad()
    def initialize_from_pretrained(self, weight: torch.Tensor):
        w_f = weight.float()
        std = torch.std(w_f, dim=-1, keepdim=True).clamp(min=1e-6)
        a0 = std * 0.4527786409
        a1 = std * 1.5104181947
        thresh = (a0 + a1) / 2.0

        abs_w = w_f.abs()
        sign_pos = (w_f >= 0)

        codes = torch.zeros_like(weight, dtype=torch.uint8)
        codes = torch.where(~sign_pos & (abs_w > thresh), torch.tensor(0, dtype=torch.uint8, device=weigh
        codes = torch.where(~sign_pos & (abs_w <= thresh), torch.tensor(1, dtype=torch.uint8, device=weig
        codes = torch.where(sign_pos & (abs_w <= thresh), torch.tensor(2, dtype=torch.uint8, device=weigh
        codes = torch.where(sign_pos & (abs_w > thresh), torch.tensor(3, dtype=torch.uint8, device=weight

        padded_k = self.packed_k * 4
        if padded_k != self.in_features:
            codes = F.pad(codes, (0, padded_k - self.in_features))

        c_reshaped = codes.view(self.out_features, -1, 4)
        packed_bytes = (
```

```
            (c_reshaped[..., 0] << 0) |
            (c_reshaped[..., 1] << 2) |
            (c_reshaped[..., 2] << 4) |
            (c_reshaped[..., 3] << 6)
        ).to(torch.uint8)

        self.packed_weights.copy_(packed_bytes)
        self.a0.copy_(a0.to(torch.float16))
        self.a1.copy_(a1.to(torch.float16))

        # Truncated SVD Residual Initialization (LoftQ)
        w_dequant = self._vectorized_dequant()
        residual = w_f - w_dequant.float()
        try:
            u, s, v = torch.svd_lowrank(residual, q=self.rank, niter=4)
            sqrt_s = torch.diag(torch.sqrt(s.clamp(min=1e-8)))
            norm_factor = 1.0 / math.sqrt(self.scaling)
            self.lora_B.copy_((u @ sqrt_s) * norm_factor)
            self.lora_A.copy_((sqrt_s @ v.t()) * norm_factor)
        except Exception:
            nn.init.kaiming_uniform_(self.lora_A, a=math.sqrt(5))
            nn.init.zeros_(self.lora_B)

    def _vectorized_dequant(self) -> torch.Tensor:
        c0 = (self.packed_weights >> 0) & 0x03
        c1 = (self.packed_weights >> 2) & 0x03
        c2 = (self.packed_weights >> 4) & 0x03
        c3 = (self.packed_weights >> 6) & 0x03

        codes = torch.stack([c0, c1, c2, c3], dim=-1).flatten(start_dim=-2)
        codes = codes[..., :self.in_features]

        w_dequant = torch.zeros(self.orig_shape, dtype=torch.float16, device=self.packed_weights.device)
        w_dequant = torch.where(codes == 0, -self.a1, w_dequant)
        w_dequant = torch.where(codes == 1, -self.a0, w_dequant)
        w_dequant = torch.where(codes == 2, self.a0, w_dequant)
        w_dequant = torch.where(codes == 3, self.a1, w_dequant)
        return w_dequant

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        w_dequant = self._vectorized_dequant().to(x.dtype)
        base_out = F.linear(x, w_dequant)
        lora_out = F.linear(F.linear(x.float(), self.lora_A), self.lora_B).to(x.dtype) * self.scaling
        out = base_out + lora_out
        if self.bias is not None:
            out = out + self.bias
        return out

    @torch.no_grad()
    def merge(self):
        """Zero-Overhead Permanent In-Situ Merge."""
        if not self.is_merged:
            delta = (self.lora_B @ self.lora_A) * self.scaling
            w_fused = self._vectorized_dequant().float() + delta
            self.initialize_from_pretrained(w_fused)
            self.lora_A.zero_()
            self.lora_B.zero_()
            self.is_merged = True
```

10.3 Native Triton Dequantization & GEMM Kernel

```
"""
M-2LRF Native Triton Dequantization & GEMM Kernel
File: m2lrf/triton_kernel.py
"""
import math
from typing import Tuple, Optional, Union
import torch
import torch.nn as nn
import torch.nn.functional as F

try:
    import triton
    import triton.language as tl
    HAS_TRITON = True
except ImportError:
    HAS_TRITON = False

if HAS_TRITON:
    @triton.jit
    def _fused_2bit_dequant_gemm_kernel(
        x_ptr, w_packed_ptr, a0_ptr, a1_ptr, out_ptr,
        M, N, K,
        stride_xm, stride_xk, stride_wn, stride_wk, stride_om, stride_on,
        BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr, BLOCK_K: tl.constexpr,
    ):
        pid_m = tl.program_id(0)
        pid_n = tl.program_id(1)
        offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
        offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
        acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)

        a0 = tl.load(a0_ptr + offs_n[:, None], mask=offs_n[:, None] < N, other=0.0)
        a1 = tl.load(a1_ptr + offs_n[:, None], mask=offs_n[:, None] < N, other=0.0)
        SUB_K: tl.constexpr = BLOCK_K // 4

        for k_iter in range(0, tl.cdiv(K, BLOCK_K)):
            k_base = k_iter * BLOCK_K
            k_sub_base = k_iter * SUB_K
            sub_idx = tl.arange(0, SUB_K)

            k0, k1, k2, k3 = k_base + sub_idx * 4 + 0, k_base + sub_idx * 4 + 1, k_base + sub_idx * 4 + 2
            x0 = tl.load(x_ptr + offs_m[:, None] * stride_xm + k0[None, :] * stride_xk, mask=(offs_m[:, N
            x1 = tl.load(x_ptr + offs_m[:, None] * stride_xm + k1[None, :] * stride_xk, mask=(offs_m[:, N
            x2 = tl.load(x_ptr + offs_m[:, None] * stride_xm + k2[None, :] * stride_xk, mask=(offs_m[:, N
            x3 = tl.load(x_ptr + offs_m[:, None] * stride_xm + k3[None, :] * stride_xk, mask=(offs_m[:, N

            k_packed = k_sub_base + sub_idx
            w_mask = (offs_n[:, None] < N) & (k_packed[None, :] < (K // 4))
            packed_bytes = tl.load(w_packed_ptr + offs_n[:, None] * stride_wn + k_packed[None, :] * strid

            c0, c1, c2, c3 = (packed_bytes >> 0) & 0x03, (packed_bytes >> 2) & 0x03, (packed_bytes >> 4) &
            v0 = tl.where(c0 == 0, -a1, tl.where(c0 == 1, -a0, tl.where(c0 == 2, a0, a1))).to(tl.float16)
            v1 = tl.where(c1 == 0, -a1, tl.where(c1 == 1, -a0, tl.where(c1 == 2, a0, a1))).to(tl.float16)
            v2 = tl.where(c2 == 0, -a1, tl.where(c2 == 1, -a0, tl.where(c2 == 2, a0, a1))).to(tl.float16)
            v3 = tl.where(c3 == 0, -a1, tl.where(c3 == 1, -a0, tl.where(c3 == 2, a0, a1))).to(tl.float16)

            acc += tl.dot(x0.to(tl.float16), tl.trans(v0))
            acc += tl.dot(x1.to(tl.float16), tl.trans(v1))
```

```
            acc += tl.dot(x2.to(tl.float16), tl.trans(v2))
            acc += tl.dot(x3.to(tl.float16), tl.trans(v3))

        out_mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
        tl.store(out_ptr + offs_m[:, None] * stride_om + offs_n[None, :] * stride_on, acc.to(tl.float16),
```

11. CONCLUSION AND OPEN RESEARCH PROBLEMS

M-2LRF demonstrates that dual-basis ternary representation combined with SVD residual initialization enables viable 2-bit quantization and parameter-efficient fine-tuning without pre-training from scratch. Future work includes bare-metal PTX assembly integration on Hopper/Blackwell Tensor Cores and dynamic activation quantization $W2A8/W2A4$.

12. APPENDIX: REPRODUCIBILITY & BENCHMARK ENVIRONMENT

To enable direct independent verification by peer researchers, the complete benchmark execution harness is made available as an interactive standalone notebook:

- **Benchmark Artifact:** `projects/m2lrf-clean/benchmarks/m2lrf_colab_benchmark.ipynb`
- **Reference Hardware:** Google Colab Cloud GPU Instance $NVIDIATeslaT4, 15.0GBVRAM, ComputeCapability7.5$.
- **Software Dependencies:** Python 3.10+, PyTorch 2.2.0+cu121, Transformers 4.40+, Datasets, Accelerate.
- **Experimental Configuration:** Random Seed = 42, Model = `gpt2` $124M$, Dataset = WikiText-2 Raw, Batch Size = 4, Sequence Length = 128, Optimizer = AdamW $\text{lr} = 2 \times 10^{-4}$, Adaptation Rank = 16, Scaling Factor $\alpha_{\text{lora}} = 16.0$.

$Endof Monograph$